

PRODUCTION OPERATIONS RUNBOOK

PostgreSQL on Kubernetes

Percona Operator — SS&C Cloud Deployment

Alert Troubleshooting & Incident Response Guide

Field	Value
Version	2.0
Platform	Kubernetes (SS&C Cloud) + Percona Operator for PostgreSQL
Monitoring	Prometheus + kube-prometheus-stack
Classification	INTERNAL — PRODUCTION USE ONLY
Maintained By	Database Platform Engineering Team
Last Updated	2026-03-07

1. Overview & Scope

This runbook provides step-by-step diagnostic and remediation procedures for the ten Prometheus alert categories observed in the production Kubernetes environment running PostgreSQL via the Percona Operator. It is designed for L1 SRE on-call engineers, L2 DBA leads, and L3 platform architects.

SCOPE

Kubernetes (SS&C Cloud) | Percona Operator for PostgreSQL | kube-prometheus-stack |
Patroni HA | pgBackRest | etcd

1.1 Environment Assumptions

- Percona PostgreSQL Operator managing StatefulSets (PostgreSQL clusters)
- Patroni handling leader election over etcd
- pgBackRest for scheduled backups (S3-compatible or NFS target)
- Prometheus + Alertmanager deployed via kube-prometheus-stack Helm chart
- SS&C Cloud Kubernetes — no public internet egress by default
- HAProxy or MetalLB for load balancing; DR cluster in separate AZ/region
- Container runtime: containerd (or CRI-O) with cgroup v2 enabled
- Node OS: RHEL 8/9 or Ubuntu 22.04 LTS with kernel 5.15+
- CNI: Cilium (observed in both production clusters)
- CSI: SS&C Cloud CSI driver (ssccloud-controller + ssccloud-node DaemonSet)

ENVIRONMENT-SPECIFIC OBSERVATIONS (SS&C Cloud)

Cluster 1 — pgkubcluster (Prod, ~401 days old):

- 3 control-plane nodes (blt6v, c2jfb, k5zhn) + 3 worker nodes (6 total)
- Namespace: postgres-operator
- PostgreSQL instances: pgkubcluster-i1-rvcz-0 (5/5), pgkubcluster-i2-d6hm-0 (4/5 — degraded)
- PgBouncer: 2 replicas via Deployment
- Percona PMM: pmm-0 in percona-pmm namespace
- Cilium CNI: 6 pods + 2 operator pods | kube-proxy: 6 pods
- ssccloud CSI: 1 controller + 2 node DaemonSet pods
- Monitoring stack age: ~16 days (recently deployed/upgraded)

Cluster 2 — salt-control-postgres (Prod, ~91 days old):

- 5 control-plane nodes (6p4tj, 8m9sf, qp7zx, wqt29, wr4gw) + 5 worker nodes (10 total)
- Namespace: postgres-operator
- PostgreSQL instances: 3 instances — i1-hpkf-0, i2-5tf6-0, i3-ptk4-0 (all 5/5)
- PgBouncer: Deployment scaled to 0/0 — **NOT RUNNING**
- Percona PMM: pmm-0 in percona-pmm namespace
- Cilium CNI: 10 pods + 2 operator pods | kube-proxy: 10 pods
- ssccloud CSI: 1 controller + 5 node DaemonSet pods
- Monitoring stack age: ~90 days

1.2 Alert Severity Reference

Severity	SLA / Response	Examples in This Runbook
CRITICAL	< 15 minutes	OOMKilled, etcd slow, StatefulSet mismatch, Backup failure
HIGH	< 30 minutes	NodeMemoryHigh, DiskIOSaturation, LB health probe
MEDIUM	< 2 hours	NodeMemoryMajorPageFaults, Node Exporter alerts
LOW	< 8 hours	Secrets expiry (advance warning)

IMPORTANT

Always start with the Quick Health Check (Section 2.3) before diving into alert-specific procedures. This establishes baseline cluster state and often reveals cascading failures.

2. Pre-Requisites & Access Setup

2.1 Required Tooling

- kubectl — configured with cluster admin or namespace-scoped DBA role
- helm — for operator and kube-prometheus-stack operations
- psql / pg_dump — PostgreSQL client tools (v14+ recommended)
- jq, curl, base64 — standard UNIX utilities
- Prometheus / Alertmanager UI or Grafana access
- pgBackRest CLI — on dedicated backup job pod or bastion
- etcdctl — v3.5+ for etcd cluster diagnostics
- stern (optional) — multi-pod log tailing for faster triage

2.2 Environment Variables (set before running commands)

```
# Set these variables at the start of every incident session
# ■■ Cluster 1: pgkubcluster ■■
export NS="postgres-operator"
export CLUSTER="pgkubcluster"

# ■■ Cluster 2: salt-control-postgres ■■
# export NS="postgres-operator"
# export CLUSTER="salt-control-postgres"

export POD=$(kubectl get pods -n $NS -l \
app.kubernetes.io/name=percona-postgresql \
--field-selector=status.phase=Running \
-o jsonpath="{.items[0].metadata.name}")

export LEADER=$(kubectl -n $NS exec -it $POD -- \
patronictl -c /etc/patroni/patroni.yaml list \
| grep Leader | awk '{print $2}')
```

ENVIRONMENT-SPECIFIC OBSERVATIONS (SS&C Cloud)

Cluster 1: NS=postgres-operator, CLUSTER=pgkubcluster

- Primary pod candidate: pgkubcluster-i1-rvcz-0 (5/5 Running)
- pgkubcluster-i2-d6hm-0 is degraded (4/5) — do NOT use as POD target

Cluster 2: NS=postgres-operator, CLUSTER=salt-control-postgres

- Primary pod candidates: i1-hpkf-0, i2-5tf6-0, i3-ptk4-0 (all 5/5)
- PgBouncer deployment is at 0/0 replicas — connection pooling is offline

2.3 Quick Health Check (run first on any alert)

Before investigating any specific alert, run this baseline health check. Cascading failures are common — an OOMKill can trigger StatefulSet mismatches, replication lag, and backup failures simultaneously.

```
# 1. Cluster pod status
kubectl get pods -n $NS -o wide

# 2. Patroni cluster state (verify leader + replicas)
kubectl -n $NS exec -it $POD -- patronictl -c /etc/patroni/patroni.yaml list

# 3. Recent events (look for warnings/errors)
kubectl get events -n $NS --sort-by=".lastTimestamp" | tail -30

# 4. Resource usage snapshot
kubectl top pods -n $NS
kubectl top nodes

# 5. Replication lag check
kubectl -n $NS exec -it $POD -- psql -U postgres -c \
"SELECT client_addr, state, sent_lsn, replay_lsn FROM pg_stat_replication;"

# 6. PgBouncer status (SS&C Cloud specific)
kubectl get deployment -n $NS | grep pgbouncer
```

2.4 Pre-Remediation Backup Procedures (MANDATORY)

Before performing ANY remediation action — patching, deleting pods, scaling, editing ConfigMaps, or rotating secrets — you **MUST** back up the relevant Kubernetes objects and PostgreSQL state. This allows rollback if a remediation step causes unintended side effects.

SAFETY PRECAUTION — BACKUP BEFORE REMEDIATION

These backup steps are **NON-NEGOTIABLE** before any remediation. Skipping these has led to data loss and extended outages in production. Create a timestamped backup directory and export all relevant objects before making changes.

Step 1 — Create timestamped backup directory:

```
export BACKUP_DIR="/tmp/k8s-backup/$(date +%Y%m%d-%H%M%S)"
mkdir -p $BACKUP_DIR
echo "Backup directory: $BACKUP_DIR"
```

Step 2 — Backup PerconaPGCluster CR (most critical):

```
kubectl -n $NS get perconapgcluster $CLUSTER -o yaml >
$BACKUP_DIR/perconapgcluster.yaml
```

Step 3 — Backup all ConfigMaps in the namespace:

```
kubectl -n $NS get configmaps -o yaml > $BACKUP_DIR/all-configmaps.yaml

# Individually back up Patroni and PG configs:
for CM in $(kubectl -n $NS get configmaps -o name | grep -E
'patroni|pgbouncer|postgres'); do
NAME=$(echo $CM | cut -d/ -f2)
kubectl -n $NS get configmap $NAME -o yaml > $BACKUP_DIR/cm-$NAME.yaml
done
```

Step 4 — Backup all Secrets (encrypted at rest):

```
kubectl -n $NS get secrets -o yaml > $BACKUP_DIR/all-secrets.yaml

# Back up specific PG/pgBackRest/TLS secrets:
for SEC in $(kubectl -n $NS get secrets -o name); do
NAME=$(echo $SEC | cut -d/ -f2)
kubectl -n $NS get secret $NAME -o yaml > $BACKUP_DIR/secret-$NAME.yaml
done
```

Step 5 — Backup StatefulSets, Deployments, Services:

```
kubectl -n $NS get statefulsets -o yaml > $BACKUP_DIR/all-statefulsets.yaml
kubectl -n $NS get deployments -o yaml > $BACKUP_DIR/all-deployments.yaml
kubectl -n $NS get services -o yaml > $BACKUP_DIR/all-services.yaml
kubectl -n $NS get endpoints -o yaml > $BACKUP_DIR/all-endpoints.yaml
```

Step 6 — Backup PersistentVolumeClaims and PersistentVolumes:

```
kubectl -n $NS get pvc -o yaml > $BACKUP_DIR/all-pvcs.yaml

# Back up the matching PVs (cluster-scoped, not namespaced):
for PVC in $(kubectl -n $NS get pvc -o
jsonpath='{.items[*].spec.volumeName}'); do
kubectl get pv $PVC -o yaml > $BACKUP_DIR/pv-$PVC.yaml
done
```

Step 7 — Backup CronJobs and Jobs:

```
kubectl -n $NS get cronjobs -o yaml > $BACKUP_DIR/all-cronjobs.yaml
kubectl -n $NS get jobs -o yaml > $BACKUP_DIR/all-jobs.yaml
```

Step 8 — Backup Pod descriptions (for debugging state):

```
for POD_NAME in $(kubectl -n $NS get pods -o name); do
NAME=$(echo $POD_NAME | cut -d/ -f2)
kubectl -n $NS describe pod $NAME > $BACKUP_DIR/describe-$NAME.txt
done
```

Step 9 — PostgreSQL logical backup (if time permits):

```
# Trigger an immediate pgBackRest backup before changes:
kubectl -n $NS exec -it $POD -- pgbackrest --stanza=db --type=diff backup

# Or dump critical metadata:
kubectl -n $NS exec -it $POD -- psql -U postgres -c \
"SELECT * FROM pg_stat_replication;" > $BACKUP_DIR/replication-state.txt

kubectl -n $NS exec -it $POD -- psql -U postgres -c \
"SELECT slot_name, active, restart_lsn FROM pg_replication_slots;" >
$BACKUP_DIR/replication-slots.txt
```

Step 10 — Archive the backup directory:

```
tar -czf $BACKUP_DIR.tar.gz -C $(dirname $BACKUP_DIR) $(basename $BACKUP_DIR)
echo "Backup archived: $BACKUP_DIR.tar.gz"
echo "Backup size: $(du -sh $BACKUP_DIR.tar.gz | cut -f1)"
```

RESTORE REFERENCE

To restore any backed-up object:

```
kubectl apply -f $BACKUP_DIR/<filename>.yaml
```

For secrets, always verify the restored secret matches the database credentials before restarting pods. For PVCs, never delete a PVC unless you have confirmed the PV reclaim policy is Retain.

3. Alert 1 — PostgreSQL Pods OOMKilled

CRITICAL

3.1 Alert Definition

Alert Name	PostgreSQL Pod OOMKilled
PromQL	kube_pod_container_status_last_terminated_reason{reason="OOMKilled", namespace=~"postgres.*"} > 0
Severity	CRITICAL
Typical Cause	Memory limit too low, runaway query, missing shared_buffers tuning
Impact	Pod restart causes connection drops; if leader killed, Patroni triggers failover

ENVIRONMENT-SPECIFIC OBSERVATIONS (SS&C Cloud)

Cluster 1 (pgkubcluster):

- pgkubcluster-i1-rvcz-0 shows **31 restarts** — strong indicator of repeated OOM or crash events
- pgkubcluster-i2-d6hm-0 is at 4/5 containers ready — one sidecar/init container not running
- StatefulSet pgkubcluster-i2-d6hm shows 0/1 READY — instance degraded

Cluster 2 (salt-control-postgres):

- salt-control-postgres-i1-hpkf-0 has 4 restarts (minor, over 90 days)
- All 3 PostgreSQL instances are 5/5 ready — healthy state

3.2 Immediate Triage

Step 1: Identify the OOMKilled pod and container:

```
kubectl get pods -n $NS -o json | jq '.items[] |
select(.status.containerStatuses[].lastState.terminated.reason=="OOMKilled")
| .metadata.name'

# SS&C Cloud specific — check both clusters:
kubectl get pods -n postgres-operator -o wide | grep -E 'pgkubcluster-i[12]'
kubectl get pods -n postgres-operator -o wide | grep -E
'salt-control-postgres-i[123]'
```

Step 2: Capture last termination details:

```
kubectl describe pod <pod-name> -n $NS | grep -A 10 "Last State"
```

Step 3: Check node-level memory pressure:

```
kubectl describe node <node-name> | grep -A 5 "Conditions"
```

Step 4: Review memory metrics via Prometheus:


```
# PromQL – container memory usage approaching limit
container_memory_working_set_bytes{namespace="postgres-operator"}
/ container_spec_memory_limit_bytes > 0.9
```

Step 5: Check cgroup memory events:

```
kubectl debug node/<node-name> -it --image=ubuntu -- \
cat /sys/fs/cgroup/kubepods/pod<pod-uid>/memory.events
```

3.3 Root Cause Identification

A. Check PostgreSQL Memory Parameters

```
kubectl -n $NS exec -it $POD -- psql -U postgres -c \
"SHOW shared_buffers; SHOW work_mem; SHOW effective_cache_size;"
```

Rule of thumb: `shared_buffers` = 25% of pod memory limit. `work_mem` x `max_parallel_workers` can OOM instantly.

B. Identify Runaway Queries

```
kubectl -n $NS exec -it $POD -- psql -U postgres -c "
SELECT pid, username, now() - query_start AS duration,
query, state, wait_event_type, wait_event
FROM pg_stat_activity
WHERE state != 'idle' ORDER BY duration DESC LIMIT 20;"
```

C. Review PostgreSQL Logs

```
kubectl -n $NS logs <pod-name> --previous --tail=200 | \
grep -iE "OOM|memory|fatal|killed"
```

D. Check Percona CR Memory Settings

```
kubectl -n $NS get perconapgcluster $CLUSTER -o yaml | grep -A 15 "resources"
```

E. Check Memory-Intensive Extensions

```
kubectl -n $NS exec -it $POD -- psql -U postgres -c \
"SELECT name, setting FROM pg_settings WHERE name IN
('shared_preload_libraries', 'max_locks_per_transaction');"

```

3.4 Remediation Steps

SAFETY PRECAUTION — BACKUP BEFORE REMEDIATION

Before patching memory limits or editing Patroni config, back up the current state:

- `kubectl -n $NS get perconapgcluster $CLUSTER -o yaml > $BACKUP_DIR/cr-before-oom-fix.yaml`
- `kubectl -n $NS get configmaps -o yaml > $BACKUP_DIR/configmaps-before-oom.yaml`
- `kubectl -n $NS get pods -o yaml > $BACKUP_DIR/pods-before-oom.yaml`
- `kubectl -n $NS exec -it $POD -- psql -U postgres -c "SHOW ALL;" > $BACKUP_DIR/pg-settings.txt`

Step 1 — Emergency: Increase Memory Limit

```
kubectl -n $NS patch perconapgcluster $CLUSTER --type=merge -p '{
  "spec": {
    "instances": [{
      "name": "instance1",
      "resources": {
        "limits": { "memory": "8Gi" },
        "requests": { "memory": "6Gi" }
      }
    }]
  }
}'
```

Step 2 — Tune PostgreSQL Parameters

```
# Edit the Percona ConfigMap or custom patroni config
kubectl -n $NS edit configmap <cluster>-patroni-config
# Recommended baseline for 8Gi pod:
shared_buffers = 2GB
work_mem = 64MB
maintenance_work_mem = 512MB
effective_cache_size = 6GB
max_connections = 200
temp_buffers = 32MB
```

Step 3 — Terminate Runaway Queries

```
kubectl -n $NS exec -it $POD -- psql -U postgres -c "
SELECT pg_terminate_backend(pid)
FROM pg_stat_activity
WHERE state != 'idle'
AND query_start < now() - interval '10 minutes';"
```

Step 4 — Verify Pod Recovers

```
kubectl get pods -n $NS -w
kubectl -n $NS exec -it $POD -- patronictl -c /etc/patroni/patroni.yaml list
```

PREVENTION

Deploy `pg_oom_adj` or use Vertical Pod Autoscaler (VPA) in recommendation mode. Set resource limits conservatively and alert at 80% memory usage. Configure `statement_timeout` and `idle_in_transaction_session_timeout`.

4. Alert 2 — NodeMemoryHighUtilization

HIGH

4.1 Alert Definition

Alert Name	NodeMemoryHighUtilization
PromQL	$(\text{node_memory_MemTotal_bytes} - \text{node_memory_MemAvailable_bytes}) / \text{node_memory_MemTotal_bytes} > 0.90$
Severity	HIGH
Threshold	> 90% for 5 minutes
Impact	Kernel OOM killer may terminate PostgreSQL or critical pods

ENVIRONMENT-SPECIFIC OBSERVATIONS (SS&C Cloud)

Cluster 1 (pgkubcluster):

- 6 nodes — higher memory pressure risk with only 3 worker nodes hosting PG
- kube-proxy pods show restarts (7, 6, 8 on some) — possible node instability

Cluster 2 (salt-control-postgres):

- 10 nodes — kube-controller-manager restarts: 5, 11, 9, 10, 5 across control planes
- kube-scheduler restarts: 4, 8, 9, 10, 2 — possible memory pressure on control planes

4.2 Immediate Triage

```
kubectl top pods --all-namespaces --sort-by=memory | head -20
kubectl top nodes

kubectl debug node/<node-name> -it --image=ubuntu
free -h
cat /proc/meminfo | grep -E "MemTotal|MemFree|MemAvailable|Cached|SwapTotal"

swapon --show
cat /proc/pressure/memory # PSI (kernel 4.20+), avg10 > 10 = contention
```

4.3 Root Cause Identification

```
# A. Top Memory Processes & Slab Cache
ps aux --sort=-%mem | head -20
cat /proc/meminfo | grep -E "Slab|SReclaimable|SUnreclaim"

# B. PostgreSQL shared_buffers vs OS Page Cache (PromQL)
node_memory_MemAvailable_bytes{instance=~"<node>"}
/ node_memory_MemTotal_bytes{instance=~"<node>"}

# C. Detect Memory Leaks in Operator or Sidecars
kubectl -n $NS top pods --containers
kubectl -n $NS get pods -o json | jq \
'.items[].status.containerStatuses[] | {name:.name,
restartCount:.restartCount}'
```

4.4 Remediation Steps

SAFETY PRECAUTION — BACKUP BEFORE REMEDIATION

Before draining nodes or modifying pod resource limits:

- `kubectl get nodes -o yaml > $BACKUP_DIR/all-nodes.yaml`
- `kubectl -n $NS get pods -o yaml > $BACKUP_DIR/pods-before-drain.yaml`
- `kubectl -n $NS get perconapgcluster $CLUSTER -o yaml > $BACKUP_DIR/cr-before-mem-fix.yaml`
- **Verify Patroni leadership and replication state before draining any node**

```
# Option A — Drop Reclaimable Kernel Caches
echo 1 > /proc/sys/vm/drop_caches # pagecache only (safest)

# Option B — Evict Non-Critical Pods
kubectl cordon <node-name>
kubectl drain <node-name> --ignore-daemonsets --delete-emptydir-data --force

# Option C — Check Connection Counts / PgBouncer
kubectl -n $NS exec -it $POD -- psql -U postgres -c \
"SELECT count(*) as active_connections FROM pg_stat_activity;"
kubectl get pods -n $NS | grep pgbouncer
```

IMPORTANT

Never enable swap on Kubernetes PostgreSQL nodes. Configure `vm.swappiness=1` on all DB nodes.

5. Alert 3 — NodeMemoryMajorPageFaults

MEDIUM

5.1 Alert Definition

Alert Name	NodeMemoryMajorPageFaults
PromQL	rate(node_vmstat_pgmajfault[5m]) > 500
Severity	MEDIUM (escalates to HIGH if sustained > 30 min)
Meaning	Pages read from disk into RAM — memory pressure indicator

A major page fault occurs when the kernel must read a page from disk. For PostgreSQL, this means data evicted from shared_buffers or OS page cache, causing latency spikes.

ENVIRONMENT-SPECIFIC OBSERVATIONS (SS&C Cloud)

Cluster 1: 3 worker nodes carry all storage I/O — page faults concentrate on fewer nodes

Cluster 2: 5 worker nodes spread the load — broader I/O distribution

5.2 Triage Steps

```
vmstat 2 10 # look for 'majflt' column
sar -B 2 10 # pgscand/s = pages scanned by kswapd
perf stat -e major-faults -p $(pgrep postgres | head -1) sleep 10

kubectl -n $NS exec -it $POD -- psql -U postgres -c "
SELECT checkpoints_timed, checkpoints_req, buffers_checkpoint,
buffers_clean, maxwritten_clean, buffers_backend
FROM pg_stat_bgwriter;"
```

5.3 Remediation

SAFETY PRECAUTION — BACKUP BEFORE REMEDIATION

Before tuning I/O parameters:

- `kubectl -n $NS get configmaps -o yaml > $BACKUP_DIR/configmaps-before-io-tune.yaml`
- `kubectl -n $NS exec -it $POD -- psql -U postgres -c "SHOW ALL;" > $BACKUP_DIR/pg-settings.txt`

```
# A. Tune PostgreSQL I/O Parameters
bgwriter_lru_maxpages = 100
bgwriter_delay = 200ms
checkpoint_completion_target = 0.9
shared_buffers = 25-40% of total pod memory
effective_io_concurrency = 200 # for SSD/NVMe

# B. Huge Pages & Swap
cat /proc/meminfo | grep Huge
# PostgreSQL: huge_pages = try
swapoff -a
```

6. Alert 4 — NodeDiskIOSaturation (dm-0)

HIGH

6.1 Alert Definition

Alert Name	NodeDiskIOSaturation
PromQL	rate(node_disk_io_time_weighted_seconds_total{device="dm-0"}[5m]) > 1
Device	dm-0 = device mapper (LVM volume backing PG data dir)
Severity	HIGH

dm-0 is a Linux device-mapper device, typically an LVM logical volume. When I/O utilization exceeds 100%, every PostgreSQL disk operation experiences latency.

ENVIRONMENT-SPECIFIC OBSERVATIONS (SS&C Cloud)

Both clusters use SS&C Cloud CSI (ssccloud) for persistent storage:

- Cluster 1: csi-ssccloud-controller (4/4) + 2 node DaemonSet pods
- Cluster 2: csi-ssccloud-controller (4/4) + 5 node DaemonSet pods
- Storage-related I/O saturation may originate from the CSI driver layer

6.2 Immediate Triage

```
iostat -x dm-0 2 10 # %util, await, svctm

kubectl -n $NS exec -it $POD -- psql -U postgres -c "
SELECT schemaname, relname, heap_blks_read, heap_blks_hit,
idx_blks_read, idx_blks_hit
FROM pg_statio_user_tables
ORDER BY heap_blks_read DESC LIMIT 20;"
```

6.3 Root Cause & Remediation

SAFETY PRECAUTION — BACKUP BEFORE REMEDIATION

Before modifying autovacuum settings, WAL config, or expanding volumes:

- `kubectl -n $NS get pvc -o yaml > $BACKUP_DIR/pvcs-before-io-fix.yaml`
- `kubectl -n $NS get perconapgcluster $CLUSTER -o yaml > $BACKUP_DIR/cr-before-io-fix.yaml`
- `kubectl -n $NS exec -it $POD -- psql -U postgres -c "SHOW ALL;" > $BACKUP_DIR/pg-settings-io.txt`
- **Trigger a pgBackRest backup before any volume expansion**


```
# A. Check Autovacuum / Bulk Operations
kubectl -n $NS exec -it $POD -- psql -U postgres -c "
SELECT pid, query_start, state, query FROM pg_stat_activity
WHERE query ILIKE '%vacuum%' OR query ILIKE '%COPY%';"

# B. Throttle Autovacuum I/O
kubectl -n $NS exec -it $POD -- psql -U postgres -c "
ALTER SYSTEM SET autovacuum_vacuum_cost_delay = '20ms';
ALTER SYSTEM SET autovacuum_vacuum_cost_limit = 200;
SELECT pg_reload_conf();"

# C. Tune WAL/Checkpoint & Expand Volume
max_wal_size = 4GB checkpoint_completion_target = 0.9
wal_compression = on synchronous_commit = local
```

BEST PRACTICE

Separate WAL (pg_wal) onto a dedicated volume. Consider io_uring (PostgreSQL 16+) for improved async I/O.

7. Alert 5 — etcdMemberCommunicationSlow

CRITICAL

CRITICAL

etcd is the backbone of Patroni HA. Slow or unreachable members cause spurious failovers, split-brain, and service disruption.

7.1 Alert Definition

PromQL	histogram_quantile(0.99, rate(etcd_network_peer_round_trip_time_seconds_bucket[5m])) > 0.15
Secondary	up{job="etcd"} == 0 (member unreachable)
Impact	Patroni leader election disrupted — potential PG downtime

ENVIRONMENT-SPECIFIC OBSERVATIONS (SS&C Cloud)

Cluster 1 — 3-member etcd: blt6v, c2jfb, k5zhn (quorum=2, tolerates 1 failure)

Cluster 2 — 5-member etcd: 6p4tj, 8m9sf, qp7zx, wqt29, wr4gw (quorum=3, tolerates 2 failures)

All etcd pods show 1/1 Running with 0 restarts in both clusters.

7.2 Immediate Triage

```
kubectl -n kube-system exec -it etcd-<node> -- \
etcdctl --endpoints=https://127.0.0.1:2379 \
--cacert=/etc/kubernetes/pki/etcd/ca.crt \
--cert=/etc/kubernetes/pki/etcd/server.crt \
--key=/etc/kubernetes/pki/etcd/server.key \
member list -w table
```

```
etcdctl endpoint health --cluster -w table
etcdctl endpoint status --cluster -w table
```

7.3 Root Cause & Remediation

SAFETY PRECAUTION — BACKUP BEFORE REMEDIATION

Before defragmenting, compacting, or removing etcd members:

- `etcdctl snapshot save $BACKUP_DIR/etcd-snapshot.db`
- `etcdctl member list -w table > $BACKUP_DIR/etcd-members.txt`
- `etcdctl endpoint status --cluster -w table > $BACKUP_DIR/etcd-status.txt`
- `kubectl -n $NS exec -it $POD -- patronictl list > $BACKUP_DIR/patroni-state.txt`
- **An etcd snapshot is critical — it is the only way to recover the entire cluster state**

```
# A. Network / Disk Latency
ping -c 20 <control-plane-node-2-ip>
iperf3 -c <target-ip> -t 30
# etcd WAL fsync latency (> 10ms p99 = problem)
histogram_quantile(0.99,
rate(etcd_disk_wal_fsync_duration_seconds_bucket[5m]))

# B. Defragment & Compact
for EP in $(etcdctl member list | awk '{print $5}'); do
etcdctl defrag --endpoints=$EP
sleep 5
done
REV=$(etcdctl endpoint status --write-out=json | jq
'.[0].Status.header.revision')
etcdctl compact $REV

# C. Restore Unhealthy Member
etcdctl member remove <member-id>
rm -rf /var/lib/etcd/member
etcdctl member add <n> --peer-urls=https://<ip>:2380

# D. Verify Patroni Stability
kubectl -n $NS exec -it $POD -- patronictl -c /etc/patroni/patroni.yaml list
```

8. Alert 6 — KubeStatefulSetReplicasMismatch

CRITICAL

ENVIRONMENT-SPECIFIC OBSERVATIONS (SS&C Cloud)

ACTIVE ISSUE — Cluster 1: StatefulSet `pgkubcluster-i2-d6hm` shows **0/1 READY**. Pod running but 4/5 containers ready.

Cluster 2: All StatefulSets healthy. **PgBouncer deployment at 0/0 replicas** — connection pooling offline.

8.1 Triage

```
kubectl get statefulsets -n $NS
kubectl describe statefulset <ss-name> -n $NS | grep -A 20 "Events"
kubectl get pods -n $NS -o wide
kubectl -n $NS exec -it $POD -- patronictl -c /etc/patroni/patroni.yaml list

kubectl logs -n $NS deployment/percona-postgresql-operator --tail=100 | \
grep -iE "error|warn|replica"
```

8.2 Root Cause

```
# Pod stuck pending — resource constraints
kubectl get pod <pending-pod> -n $NS -o yaml | grep -A 20 "conditions"

# PVC not bound
kubectl get pvc -n $NS
kubectl describe pvc <pvc-name> -n $NS

# Image pull / init container failures
kubectl describe pod <pod> -n $NS | grep -iE "pull|image|registry"

# Check individual container status (5 containers per PG pod)
kubectl get pod <pod-name> -n $NS -o
jsonpath='{.status.containerStatuses[*].name}'
kubectl get pod <pod-name> -n $NS -o
jsonpath='{.status.containerStatuses[*].ready}'
```

8.3 Remediation

SAFETY PRECAUTION — BACKUP BEFORE REMEDIATION

Before deleting pods, re-initializing replicas, or scaling PgBouncer:

- `kubectl -n $NS get statefulsets -o yaml > $BACKUP_DIR/statefulsets-before-fix.yaml`
- `kubectl -n $NS get pods -o yaml > $BACKUP_DIR/pods-before-fix.yaml`
- `kubectl -n $NS get pvc -o yaml > $BACKUP_DIR/pvcs-before-fix.yaml`
- `kubectl -n $NS get deployments -o yaml > $BACKUP_DIR/deployments-before-fix.yaml`
- `kubectl -n $NS describe pod <stuck-pod> > $BACKUP_DIR/describe-stuck-pod.txt`
- **Confirm Patroni leader/replica state before deleting any pod**

```
# Fix A — Delete stuck pod (StatefulSet recreates it)
kubectl delete pod <stuck-pod> -n $NS

# Fix B — Force Patroni re-initialize replica
kubectl -n $NS exec -it $POD -- \
patronictl -c /etc/patroni/patroni.yaml reinit $CLUSTER <replica>

# Fix C — For PgBouncer deployment at 0 replicas (Cluster 2):
kubectl -n $NS scale deployment $CLUSTER-pgbouncer --replicas=2
```

9. Alert 7 — Node Exporter Alerts

MEDIUM

9.1 Common Alerts

Alert	Threshold	Typical Cause
NodeCPUHighUsage	> 90% for 15m	Runaway query, parallel query overload
NodeFileSystemAlmostOutOfSpace	< 10% free	WAL bloat, temp files, core dumps
NodeFileSystemFilesFillingUp	< 5% inodes	Many small temp files, WAL segments
NodeNetworkReceiveErrors	> 0.01/s	NIC driver issues, MTU mismatch
TargetDown	up == 0	node-exporter crashed / DaemonSet issue

ENVIRONMENT-SPECIFIC OBSERVATIONS (SS&C Cloud)

Cluster 1: node-exporter: 6/6 running, kube-prometheus-stack ~16 days old

Cluster 2: node-exporter: 10/10 running (some with restarts: 1-4), stack ~90 days old

9.2 Triage & Fixes

SAFETY PRECAUTION — BACKUP BEFORE REMEDIATION

Before cleaning WAL files, dropping replication slots, or modifying DaemonSets:

- `kubectl -n $NS exec -it $POD -- psql -U postgres -c "SELECT * FROM pg_replication_slots;" > $BACKUP_DIR/repl-slots.txt`
- `kubectl get daemonsets -n monitoring -o yaml > $BACKUP_DIR/daemonsets-monitoring.yaml`
- **Never drop an active replication slot — verify slot is inactive first**

```
# DaemonSet & pod health
kubectl get daemonset -n monitoring | grep node-exporter
kubectl get pods -n monitoring -l app.kubernetes.io/name=node-exporter -o wide
kubectl logs -n monitoring <node-exporter-pod> --tail=50

# Check taints blocking DaemonSet
kubectl describe node <node-name> | grep Taint

# FileSystem Out of Space
kubectl debug node/<node-name> -it --image=ubuntu
du -sh /host/var/lib/postgresql/data/* 2>/dev/null | sort -rh | head -20

# Clean old WAL — verify replication slots first
kubectl -n $NS exec -it $POD -- psql -U postgres -c "
SELECT slot_name, active, restart_lsn FROM pg_replication_slots;"
```

10. Alert 8 — Load Balancer Health Probe Issues (DR)

HIGH

ENVIRONMENT-SPECIFIC OBSERVATIONS (SS&C Cloud)

Cluster 1 — Service Topology:

- pgkubcluster-pgbouncer: NodePort 10.105.44.229:5432:30543/TCP
- pgkubcluster-primary: ClusterIP (headless), 5432/TCP
- pgkubcluster-replicas: ClusterIP 10.109.229.130, 5432/TCP
- PgBouncer: 2 replicas running

Cluster 2 — Service Topology:

- salt-control-postgres-ha: NodePort 10.106.107.207:5432:30543/TCP
- salt-control-postgres-pgbouncer: NodePort 10.107.93.161:5432:32005/TCP
- salt-control-postgres-replicas: ClusterIP 10.111.190.36, 5432/TCP
- **PgBouncer 0/0 — NodePort 32005 has no backend pods!**

10.1 Triage

```
kubectl get svc,Endpoints -n $NS
kubectl run test-conn --image=postgres:14 --rm -it -- \
pg_isready -h <service-ip> -p 5432 -U postgres
kubectl -n $NS exec -it $POD -- \
curl -s http://localhost:8008/health | python3 -m json.tool
kubectl -n $NS exec -it $POD -- patronictl -c /etc/patroni/patroni.yaml list
kubectl get networkpolicy -n $NS
```

10.2 Remediation

SAFETY PRECAUTION — BACKUP BEFORE REMEDIATION

Before modifying services, endpoints, or scaling PgBouncer:

- `kubectl -n $NS get svc -o yaml > $BACKUP_DIR/services-before-lb-fix.yaml`
- `kubectl -n $NS get endpoints -o yaml > $BACKUP_DIR/endpoints-before-lb-fix.yaml`
- `kubectl -n $NS get networkpolicy -o yaml > $BACKUP_DIR/netpol-before-lb-fix.yaml`
- `kubectl -n $NS get deployments -o yaml > $BACKUP_DIR/deployments-before-lb-fix.yaml`

```
# DR replication check
kubectl -n $NS exec -it $POD -- psql -U postgres -c "
SELECT client_addr, state, sent_lsn, replay_lsn,
(sent_lsn - replay_lsn) AS replication_lag
FROM pg_stat_replication;"

# Fix PgBouncer if down (Cluster 2 specific):
kubectl -n $NS scale deployment $CLUSTER-pgbouncer --replicas=2
```


11. Alert 9 — Backup Job Failures

CRITICAL

RPO RISK

A failed backup means your Recovery Point Objective is at risk. Escalate to L2 DBA immediately if manual retry also fails.

ENVIRONMENT-SPECIFIC OBSERVATIONS (SS&C Cloud)

Cluster 1 — Backup Status:

- CronJob: pgkubcluster-pgbackupmonitor, schedule 50 23 * * *
- **FAILED jobs:** 29518550(19d), 29519990(18d), 29521430(17d)
- Recent jobs recovered: 29543030, 29544470, 29545910 (Complete)

Cluster 2 — Backup Status:

- CronJob: salt-control-postgres-pgbackupmonitor, schedule 50 23 * * *
- All jobs Complete, durations 4m56s-7m18s — **HEALTHY**

11.1 Triage

```
kubectl get jobs -n $NS | grep -i backup
kubectl describe job <backup-job> -n $NS
kubectl -n $NS exec -it $POD -- pgbackrest --stanza=db info
kubectl -n $NS exec -it $POD -- cat /var/log/pgbackrest/backup.log | tail -100
```

11.2 Root Cause & Fixes

SAFETY PRECAUTION — BACKUP BEFORE REMEDIATION

Before modifying backup stanzas, removing locks, or re-creating repos:

- kubectl -n \$NS get cronjobs -o yaml > \$BACKUP_DIR/cronjobs-before-backup-fix.yaml
- kubectl -n \$NS get jobs -o yaml > \$BACKUP_DIR/jobs-before-backup-fix.yaml
- kubectl -n \$NS get secrets -o yaml > \$BACKUP_DIR/secrets-before-backup-fix.yaml
- kubectl -n \$NS exec -it \$POD -- pgbackrest --stanza=db info > \$BACKUP_DIR/pgbackrest-info.txt
- kubectl -n \$NS exec -it \$POD -- psql -U postgres -c "SELECT * FROM pg_stat_archiver;" > \$BACKUP_DIR/archiver-state.txt

```
# A. Repository connectivity
kubectl -n $NS exec -it $POD -- pgbackrest info --log-level-console=detail
kubectl get secret -n $NS | grep pgbackrest

# B. Stanza init
kubectl -n $NS exec -it $POD -- pgbackrest --stanza=db stanza-create
kubectl -n $NS exec -it $POD -- pgbackrest --stanza=db check

# C. WAL archive check
kubectl -n $NS exec -it $POD -- psql -U postgres -c "
SELECT archived_count, failed_count, last_failed_wal
FROM pg_stat_archiver;"

# D. Remove stale lock + manual backup
kubectl -n $NS exec -it $POD -- rm -f /tmp/pgbackrest/db-backup.lock
kubectl -n $NS exec -it $POD -- pgbackrest --stanza=db --type=full backup
```

BEST PRACTICE

Retention: full=2, diff=7. Schedule full weekly, differential daily, WAL continuously. Test restores quarterly.

12. Alert 10 — API Keys / Secrets Expiry

LOW

Escalates to HIGH (< 7 days), CRITICAL (expired).

12.1 Audit Secrets

```
kubectl get secrets -n $NS \
-o custom-columns=NAME:.metadata.name,TYPE:.type,CREATED:.metadata.creationTimestamp

# TLS cert expiry check
for SECRET in $(kubectl get secrets -n $NS \
-o jsonpath='{.items[?(@.type=="kubernetes.io/tls")].metadata.name}'); do
echo "=== $SECRET ==="
kubectl get secret $SECRET -n $NS -o jsonpath='{.data.tls\.crt}' \
| base64 -d | openssl x509 -noout -enddate
done
```

ENVIRONMENT-SPECIFIC OBSERVATIONS (SS&C Cloud)

- Cluster 1 age: ~374 days — secrets may be approaching expiry
- Cluster 2 age: ~90 days — secrets are relatively fresh
- Check Percona PMM credentials in percona-pmm namespace as well

12.2 Rotation Procedures

SAFETY PRECAUTION — BACKUP BEFORE REMEDIATION

Before rotating passwords or deleting TLS secrets:

- `kubectl -n $NS get secrets -o yaml > $BACKUP_DIR/all-secrets-before-rotation.yaml`
- `kubectl -n $NS get perconapgcluster $CLUSTER -o yaml > $BACKUP_DIR/cr-before-rotation.yaml`
- **Record the current password before overwriting — store in the backup directory**
- **Verify application connection strings are updated after rotation**

```
# PostgreSQL password rotation
NEW_PASS=$(openssl rand -base64 32)
kubectl create secret generic <pg-secret> -n $NS \
--from-literal=password=$NEW_PASS --dry-run=client -o yaml | kubectl apply -f
-

kubectl -n $NS exec -it $POD -- psql -U postgres -c \
"ALTER USER postgres PASSWORD '$NEW_PASS';"

kubectl -n $NS annotate perconapgcluster $CLUSTER
percona.com/issue-restart="$(date +%s)"

# TLS cert renewal (cert-manager)
kubectl delete secret <tls-secret> -n $NS # cert-manager re-creates
```

12.3 Proactive Monitoring

```
apiVersion: monitoring.coreos.com/v1
kind: PrometheusRule
spec:
  groups:
    - name: certificate-expiry
  rules:
    - alert: TLSCertExpiringSoon
      expr: x509_cert_expiry - time() < 30 * 86400
      labels: {severity: warning}
    - alert: TLSCertCritical
      expr: x509_cert_expiry - time() < 7 * 86400
      labels: {severity: critical}
```

13. Escalation Matrix

Alert	L1 (SRE On-Call)	L2 (DBA Lead)	L3 (Platform Arch)
OOMKilled	Restart pod, check mem	Tune PG params, raise limits	Capacity planning, VPA
NodeMemoryHigh	Cordon/drain node	Identify leak, tune buffers	Node pool expansion
DiskIOSaturation	Cancel vacuum, check WAL	I/O tuning, pg_wal sep.	Storage architecture
etcd Slow/Down	Check pods, defrag	Member recovery	etcd cluster rebuild
StatefulSet Mismatch	Delete stuck pod	Patroni reinit	Operator upgrade
Backup Failure	Check logs, retry	Fix repo, restore test	RPO breach notify
Secrets Expiry	Rotate, restart pods	Audit all creds	Vault integration
LB / DR Issues	Check endpoints	Replication check	DR arch review

ESCALATION GUIDELINES

L1 escalates to L2 after 15 min without resolution for CRITICAL, 30 min for HIGH. L2 escalates to L3 if root cause unidentified within 1 hour. Update incident channel every 15 min.

14. Post-Incident Checklist

14.1 Immediate Recovery Verification (within 1 hour)

- All PostgreSQL pods Running and Ready
- Patroni: exactly one Leader, all configured replicas Streaming
- Backup job executed and completed successfully after incident
- etcd cluster healthy (all members reachable, no pending elections)
- Prometheus targets all UP (node-exporter, postgres-exporter)
- Replication lag < 1 second on all standbys
- No active OOMKilled pods or CrashLoopBackOff
- DR site replication current and verified
- Client applications reconnected and healthy
- PgBouncer deployment running with expected replica count

14.2 Verification Commands

```
# Full health snapshot
kubectl get pods,statefulsets,pvc,svc -n $NS
kubectl -n $NS exec -it $POD -- patronictl list
kubectl -n $NS exec -it $POD -- psql -U postgres -c \
"SELECT * FROM pg_stat_replication;"
kubectl -n $NS exec -it $POD -- pgbackrest --stanza=db info
kubectl top nodes && kubectl top pods -n $NS

# Verify PgBouncer (SS&C Cloud specific)
kubectl get deployment -n $NS | grep pgbouncer
kubectl get endpoints -n $NS | grep pgbouncer
```

14.3 Post-Incident Review Actions

- Document incident timeline in your incident tracker
- Capture Prometheus metrics screenshots (memory, I/O, replication lag)
- Update this runbook with any new findings or improved commands
- File a Jira/ticket for permanent fixes (resource limits, tuning)
- Schedule postmortem within 48 hours for P1 incidents
- Review Alertmanager thresholds — alert fire too late or early?
- Validate DR replication is current and DR cluster can promote
- Review and update capacity forecasts based on incident data
- Verify monitoring dashboards captured the incident timeline
- Clean up backup directory — archive to shared storage and remove from /tmp

IMPORTANT

Every CRITICAL incident should have a 5-Why root cause analysis documented within 5 business days. This prevents recurrence and improves runbook quality. Assign an owner for each action item with a due date.

— END OF RUNBOOK —

PostgreSQL on Kubernetes — Percona Operator — SS&C Cloud Deployment — Production Runbook v2.0